ROLLING DEPLOY DROPS REQUESTS?

Kubernetes graceful shutdown,
explained in 10 slides.

502 · 502 · 502

@k4coding

THE SYMPTOM

Every deploy, users get a few seconds of 502 errors or connection resets.

Readiness probes are set. The new pods are healthy. The errors still come back on the next rollout.

The problem is not the new pods. It is how the old pods die.

TWO THINGS, AT ONCE

When a pod is deleted, Kubernetes starts two things in parallel:

1. The kubelet begins stopping the container.

2. The pod is removed from Service endpoints, and kube-proxy, ingress and load balancers catch up.

Step 2 takes time to reach every node. Until it does, traffic can still be sent to a pod that is shutting down.

THE SHUTDOWN ORDER

preStop
hook runs first, if you set one

SIGTERM
is sent to PID 1 in the container

Grace period
terminationGracePeriodSeconds, default 30 s

SIGKILL
whatever is still running is killed

BUG 1: INSTANT EXIT

Many apps exit the moment they get SIGTERM.

In-flight requests are cut off halfway.

New requests that are still routed to the pod hit a closed port.

Result: errors on every rollout, even though nothing is broken.

BUG 2: SHELL AS PID 1

Dockerfile

```
CMD npm start
# shell form: sh is PID 1
```

With shell form, a shell can end up as PID 1, and it does not forward SIGTERM to your app.

Your app never hears the signal, waits out the 30 s grace period, then gets SIGKILL.

FIX 1: PRESTOP PAUSE

deployment.yaml

```yaml
lifecycle:
  preStop:
    exec:
      command: ["sleep","10"]
```

A short pause before SIGTERM gives endpoints and load balancers time to stop sending traffic. The image needs a sleep binary; newer Kubernetes versions also offer a built-in sleep action.

FIX 2: HANDLE SIGTERM

Dockerfile

```
CMD ["node", "server.js"]
# exec form: app is PID 1
```

- ✓ Use exec form so the app gets the signal
- ✓ On SIGTERM, stop accepting new connections
- ✓ Finish in-flight requests
- ✓ Close DB and queue connections, then exit

FIX 3: ENOUGH TIME

deployment.yaml

```
terminationGracePeriodSeconds: 45
```

The grace period covers the preStop hook AND the app's drain time.

Rule of thumb: grace period > preStop pause + your longest request.

Use case: an API with 20 s report exports, 10 s preStop and 45 s grace ships every deploy with zero 502s.

SUMMARY

- ✓ Pods keep getting traffic for a few seconds after shutdown starts
- ✓ Add a short preStop pause
- ✓ Use exec form and handle SIGTERM
- ✓ Drain in-flight work, then exit
- ✓ Grace period > preStop + longest request

Follow for one DevOps fix every day